

목차

페이지	내용
1부 · C언어 전체 한권 (기본 문법)	
3	프로그램 구조 · 전처리 · 컴파일 · 링크
4	자료형 크기와 범위
5	리터럴 · 서식
6	scanf와 이스케이프
7	연산자
8	제어문
9	함수
10	포인터 · 구조체 · 리스트 · 큐 · GCC
2부 · C언어 학습 보충 노트	
11	리터럴 · 서식 지정자 · 이스케이프
12	보충삽입 A - printf / 이스케이프 / unsigned
13-14	연산자 우선순위 · 형 변환 · while/do-while · break/continue
15	배열과 문자열
16	문자열 함수와 배열 · 포인터
17	보충삽입 B - parameter / argument 구분
18-19	포인터와 메모리
20	연결 리스트 완전 정리
21-22	큐와 환형 큐
23-24	표준 키워드 전체 지도
3부 · C언어 종합 정리 노트	
25	이중 포인터 · 배열-포인터 인덱스 · NULL/범위 · C vs C++
26	자주 틀리는 코드 패턴 · 시험용 압축 암기표
27	포인터 · 연산자 · 제어문 압축표
4부 · C언어 최종 핵심 보충노트	
28	01 선언 / 정의 / 초기화 / 대입
29	02 값 범주와 형 변환
30	03 const / static / extern
31	04 메모리의 실제 모습
32	05 동적 메모리 안전 규칙
33	06 함수 포인터와 콜백
34	07 재귀와 호출 스택
35	08 argc / argv
36	09 파일 입출력

페이지	내용
37	10 오류 처리와 방어적 C
38	11 비트 마스크 실전
39	12 헤더와 다중 파일 프로그램
40	13 컴파일러 경고와 디버깅
41	보충삽입 C - GDB watch 실제 사용 예
42	14 Undefined Behavior와 흔한 사고
43	15 검색·정렬의 표준 도구
44	16 문제 풀이용 실전 패턴
45	초압축 레퍼런스
5부 · C언어 자료구조 4종 핵심 총정리	
46	개요 (Binary Tree · BST · Linked List · Stack & Queue)
47	전체 구성
48	01 문제 풀이 전 C 핵심 복습
49	02 포인터와 구조체: 자료구조의 언어
50	03 동적 메모리와 수명 관리
51-52	04 Linked List
53	05 Stack
54	06 Queue / Circular Queue
55-56	07 Binary Tree
57-58	08 Binary Search Tree (BST)
59	09 재귀와 순회 패턴
60	10 시간복잡도와 문제 읽는 법
61	11 자주 터지는 오류와 디버깅
62-63	12 실전 구현 템플릿 모음
6부 · C언어 보충 통합본 (13단원 빈칸 + 구현 부록)	
64-65	1부 13단원 빈칸 (기본문법 ~ 파일입출력)
66-67	2부 서식 · 헤더 · 변환 (strtol / assert)
68-71	3부 구현 뼈대 (리스트 · 환형 · 스택 · 큐 · BST · Makefile)
7부 · 보충 페이지	
72	BST 삭제 · Makefile (이어서)
73	환형 단일 연결 리스트
74	환형 이중 연결 리스트
75	환형 큐 (원형 큐)
76	C에서 NaN과 INF

프로그램 구조

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

#include는 전처리, main은 시작점, printf는 라이브러리 호출, return 0은 정상 종료를 운영체제에 알린다.

전처리 · 컴파일 · 링크

단계	하는 일	결과
전처리	#include, #define 처리	전처리된 소스
컴파일	문법 검사 후 목적 코드로 변환	object 파일
링크	목적 코드와 라이브러리 연결	실행 파일
실행	운영체제가 실행 파일을 적재	프로그램 실행
gcc main.c -o main		

실무에서 “컴파일했다”는 말은 전처리와 링크까지 포함한 경우가 많다. 컴파일 오류와 링크 오류는 다른 단계다.

자료형 크기와 범위

기준은 일반적인 64비트 환경이다. long과 포인터 크기는 OS/컴파일러에 따라 달라질 수 있다.

Windows 64비트(MSVC, LLP64)에서 long은 4바이트, Linux 64비트(LP64)에서 long은 8바이트다. C99의 논리형은 _Bool이며 <stdbool.h>로 bool / true / false를 쓴다.

자료형	설명	크기	범위	포인트
bool / _Bool	논리형	보통 1 byte	true, false	stdbool.h
char	문자 / 1바이트 정수	1 byte	signed -128 ~ 127 unsigned 0 ~ 255	'A' 는 문자 하나
short	작은 정수	2 bytes	-32,768 ~ 32,767	short a = 100;
int	기본 정수	4 bytes	-2,147,483,648 ~ 2,147,483,647	가장 많이 씀
unsigned int	음수 없는 정수	4 bytes	0 ~ 4,294,967,295	언더플로 주의
long	정수	Win64 4 / Linux64 8	크기에 따라 다름	100L
long long	큰 정수	8 bytes	약 $\pm 9.22 \times 10^{18}$	100LL
float	단정밀 실수	4 bytes	약 $\pm 3.4 \times 10^{38}$ 유효숫자 6~7자리	3.14f
double	배정밀 실수	8 bytes	약 $\pm 1.7 \times 10^{308}$ 유효숫자 15~16자리	3.14
long double	확장 실수	8 bytes 이상	환경마다 다름	3.14L
포인터	주소	64비트에서 보통 8	주소값	가리키는 형과 크기 무관

리터럴

적은 값	의미
'A'	문자 1개
"ABC"	문자열 (끝 \0)
10	int
3.14	double
3.14f	float
100L	long
100LL	long long
10u	unsigned int

서식

서식	대상
%d / %i	int
%u	unsigned int
%ld / %lld	long / long long

서식	대상
%f	double (printf)
%c	문자
%s	문자열
%p	포인터
%x / %o	16진수 / 8진수
%zu	size_t / sizeof
<pre>unsigned int u = 0; u--; /* 32-bit: 4294967295 */ printf("%zu\n", sizeof(int)); printf("%o\n", 8); /* 10 */</pre>	

scanf와 이스케이프

```
int n;
scanf("%d", &n);

/* 문자열 배열 이름은 주소이므로 &를 붙이지 않는다 */
char name[20];
scanf("%19s", name);
```

%19s는 최대 19글자 + 마지막 \0을 위한 공간이다.

표현	뜻
\n	줄바꿈
\t	탭
\r	줄의 처음으로 이동
\b	백스페이스
\\	역슬래시
\0	문자열 끝 NUL
<pre>printf("안녕\n하세요\n"); printf("이름:\t홍길동\n");</pre>	

연산자

```
printf("%d\n", 5/2); /* 2 */  
printf("%f\n", 5.0/2); /* 2.000000 */
```

```
if (age >= 14 && score >= 60) {  
    printf("합격");  
}
```

```
int a = 5;  
int b = ++a; /* 먼저 증가 -> a=6, b=6 */  
a = 5;  
b = a++; /* 나중에 증가 -> b=5, 그 다음 a=6 */  
int max = (a > b) ? a : b;
```

제어문

```
if (score >= 90) {  
    printf("A");  
} else if (score >= 80) {  
    printf("B");  
} else {  
    printf("C 이하");  
}  
  
switch (menu) {  
    case 1: printf("시작"); break;  
    case 2: printf("종료"); break;  
    default: printf("잘못된 메뉴");  
}  
  
for (초기식; 조건식; 증감식) {  
    본문;  
}
```

if (x = 3)은 대입이다. 비교는 if (x == 3)이다. switch에서 break가 없으면 fall-through가 난다.

함수

```
int add(int a, int b);    /* 선언 */
int add(int a, int b) {   /* 정의 */
    return a + b;
}
int x = add(2, 3);        /* 호출 */

void change(int x) { x = 100; }
void change2(int *x) { *x = 100; }
change(a);    /* a는 그대로 */
change2(&a);  /* a가 바뀜 */

void count(void) {
    static int n = 0;
    printf("%d\n", ++n);
}
```

포인터 · 구조체 · 리스트 · 큐 · GCC

```
int *p = NULL;
if (p != NULL) printf("%d", *p);

int *a = malloc(5 * sizeof *a);
if (a == NULL) { /* 할당 실패 */ }
free(a);
a = NULL;

printf("%d", p->id); /* (*p).id 와 같다 */

if (prev == NULL) ll->head = cur->next;
else prev->next = cur->next;
free(cur);

gcc main.c -o main
gcc -Wall -Wextra main.c -o main
gcc -g main.c -o main
```

C언어 기초 표현 ① - 리터럴·서식 지정자·이스케이프

값을 어떻게 적고, printf/scanf가 그것을 어떻게 해석하는지

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 리터럴

리터럴이란?

소스 코드에 직접 적어 놓은 "값 그 자체"입니다. 10, -3, 3.14, 'A', "Hello" 같은 것이 대표적입니다. 변수 이름은 값의 이름이지 리터럴이 아닙니다.

종류	예	의미
정수 리터럴	10, -3, 0x10, 077	정수 값. 0x는 16진수, 앞의 0은 8진수 표기로 쓰일 수 있음
실수 리터럴	3.14, 1.0e-3	부동소수점 값
문자 리터럴	'A', '\n'	문자 하나. C에서는 정수형과 밀접하게 취급됨
문자열 리터럴	"Hello"	문자 배열처럼 저장되며 끝에 \0이 붙음

주의: 문자 'A'와 문자열 "A"는 다릅니다. 'A'는 문자 하나를 나타내는 문자 상수이고, "A"는 A와 문자열 종료 문자 \0을 가진 문자열입니다.

2. printf 서식 지정자

printf는 값 자체만 출력하는 함수가 아니라, 첫 번째 문자열 안의 서식 지정자와 뒤의 인자를 대응시켜 출력합니다.

서식	대표 대상	예
%d / %i	int	printf("%d", n);
%u	unsigned int	printf("%u", n);
%ld / %lu	long / unsigned long	길이가 긴 정수
%f	double (printf에서)	printf("%.2f", x);
%c	char가 promoted 된 int	문자 출력

보충 삽입 A - printf / 이스케이프 / unsigned

이번 대조에서 추가된 누락 내용

%o : unsigned int을 8진수로 출력

%zu : size_t 출력. sizeof 결과에 적합

```
printf("%zu\n", sizeof(int)); // sizeof returns size_t
printf("%o\n", 8);           // 10 in octal
```

이스케이프 문자 예제도 함께 기억한다. \wr은 현재 줄의 맨 앞으로 이동하고, \wb는 백스페이스, 작은따옴표는 \'로 표현한다.

```
printf("A\rB\n"); // carriage return example
```

unsigned는 음수를 표현하지 않는 대신 양의 범위를 넓힌다. 0에서 1을 빼는 언더플로 예시는 다음과 같다.

```
unsigned int u = 0;
u--; // 32-bit unsigned example: 4294967295
```

따라서 음수가 나올 수 있는 계산에 unsigned를 섞으면 매우 큰 값으로 보이는 결과가 생길 수 있다.

높은 편	대표 연산
높음	() 함수호출, [] 배열/인덱싱, 단항 연산
중간	*, /, %
중간	+, -
비교	< <= > >=
동등	== !=
논리	&&,
낮음	조건 ? :, 대입 = += -= 등

모든 연산자 우선순위를 외우기보다 "복잡해지면 괄호"라는 원칙과 자주 쓰는 순서를 익히는 편이 실전에서 안전합니다.

8. 형 변환

```
int a=5, b=2;
double x = (double)a / b; // 2.5
```

(double)a 같은 명시적 형 변환은 계산 전에 a를 double로 바꿉니다. 반면 int a=5; int b=2;에서 a/b는 먼저 정수 나눗셈이 되어 2가 됩니다.

4. while / do-while

```
while (count < 5) {  
    count++;  
}  
  
do {  
    scanf("%d", &n);  
} while (n != 0);
```

while은 조건을 먼저 검사하므로 0회 실행될 수 있습니다. do-while은 먼저 한 번 실행한 뒤 조건을 검사하므로 최소 1회 실행됩니다.

5. break와 continue

```
for (int i=0; i<10; i++) {  
    if (i == 3) continue;  
    if (i == 7) break;  
    printf("%d ", i);  
}
```

continue는 현재 반복의 나머지를 건너뛰고 다음 반복으로, break는 반복문 자체를 빠져나옵니다.

중첩 반복문에서 break는 가장 가까운 반복문 하나만 종료합니다. 바깥 루프까지 나가는 것은 아닙니다.

6. 무한 루프와 안전한 종료

```
while (1) {  
    scanf("%d", &c);  
    if (c == 0) break;  
}
```

메뉴 프로그램에서 흔히 쓰이는 형태입니다. "종료 조건을 어디서 검사하는지"가 명확해야 합니다.

C언어 ④ - 배열과 문자열

1차원·2차원 배열, 인덱스, 초기화, 문자열, 배열과 포인터의 관계

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 배열 기본

```
int a[5];
int b[5] = {1,2,3,4,5};
int c[] = {10,20,30};
```

배열은 같은 형의 데이터를 연속된 메모리 공간에 여러 개 저장합니다. 인덱스는 0부터 시작합니다. int a[5]의 유효한 인덱스는 0~4입니다.

a[5]에 접근하면 배열의 범위를 벗어납니다. 컴파일러가 항상 즉시 잡아내는 것이 아니므로 특히 위험합니다.

2. 2차원 배열

```
int score[2][3] = {
    {10,20,30},
    {40,50,60}
};
printf("%d", score[1][2]); // 60
```

2차원 배열은 "행과 열"로 생각하면 쉽습니다. 메모리에는 행 단위로 연속해서 저장됩니다.

3. sizeof와 배열

```
int a[5];
size_t bytes = sizeof(a);
size_t count = sizeof(a) / sizeof(a[0]);
```

배열이 실제 배열로 존재하는 그 범위에서는 sizeof(a)는 전체 배열의 크기이고 sizeof(a[0])은 한 원소의 크기입니다. 둘을 나누면 원소 개수를 구할 수 있습니다.

함수 매개변수로 전달된 배열은 실제로 포인터처럼 조정되기 때문에 sizeof(param)이 전체 배열 크기를 주지 않는 경우가 많습니다. 함수에 배열과 길이를 함께 전달하는 이유가 이것입니다.

4. 문자열

```
char s[6] = "Hello"; // H e l l o \0

printf("%s", s);
```

C 문자열은 별도의 string 타입이 아니라 char 배열에 저장하며, 마지막에 NUL 문자 \0이 있어야 문자열 함수가 끝을 알 수 있습니다.

함수	역할
strlen	문자열 길이 계산(₩0 제외)
strcpy	문자열 복사
strcmp	문자열 비교
strcat	문자열 이어 붙이기
문자열 함수는 목적지 배열의 충분한 공간이 있다는 전제가 중요합니다. 버퍼 크기와 경계 검사를 항상 생각해야 합니다.	

5. 배열과 포인터

```
int a[3] = {10, 20, 30};
int *p = a;
printf("%d", *(p+1)); // 20
```

배열 이름 `a`는 많은 표현식에서 첫 원소의 주소처럼 변환됩니다. `p+1`은 "1바이트 뒤"가 아니라 `int` 한 개 크기만큼 다음 원소를 가리킵니다.

6. 함수에 배열 전달

```
void printArray(const int a[], int n) {
    for (int i=0; i<n; i++)
        printf("%d ", a[i]);
}
```

C에서는 배열 자체를 함수 인자로 통째로 복사하지 않습니다. 함수가 원본 배열을 읽거나 수정할 수 있도록 주소와 길이를 사용하는 방식이 기본입니다.

보충 삽입 B - parameter / argument 명확히 구분

이번 대조에서 추가된 누락 내용

함수 관련 용어를 정확히 구분하면 코드를 읽을 때 혼동이 줄어든다.

용어	의미
매개변수 (parameter)	함수 정의에서 값을 받는 변수
인자 (argument)	함수 호출 때 실제로 전달하는 값
반환값 (return value)	함수가 호출한 곳으로 돌려주는 값
void	반환값이 없음을 나타냄

```
int add(int a, int b) {  
    return a + b;  
}  
int r = add(2, 3);
```

여기서 'a', 'b'는 매개변수이고 '2', '3'은 인자다. 함수 정의와 호출을 기준으로 구분하면 된다.

C언어 ⑥ - 포인터와 메모리

주소(&), 역참조(*), 포인터 연산, 배열 관계, 이중 포인터, NULL, 동적 메모리

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 주소와 포인터

```
int value = 10;
int *p = &value;
printf("%d", *p);
```

&value는 value가 저장된 메모리의 주소이고, p는 그 주소를 저장하는 포인터입니다. *p는 "p가 가리키는 곳의 값"을 의미합니다.

표현	뜻
&x;	x의 주소
p	포인터 p에 저장된 주소값 자체
*p	p가 가리키는 메모리의 값
int *p	int를 가리키는 포인터 선언

2. 포인터 연산

```
int a[3] = {10, 20, 30};
int *p = a;

printf("%d", *(p+1)); // 20
```

p+1은 주소 숫자를 1만큼 증가시키는 개념이 아니라, 가리키는 형의 크기만큼 다음 원소로 이동하는 포인터 연산입니다. int*라면 sizeof(int)만큼 이동하는 효과를 냅니다.

포인터에 정수 1을 더하는 것과 포인터가 가리키는 값에 1을 더하는 것은 완전히 다릅니다. p+1과 (*p)+1을 구분하세요.

3. 포인터와 scanf

```
int n;
scanf("%d", &n);

int *p = &n;
*p = 50;
```

scanf는 변수 자체가 아니라 "어디에 저장할지"를 알아야 하므로 주소를 전달합니다. *p=50은 그 주소가 가리키는 실제 변수 n의 값을 50으로 바꾸는 것입니다.

```
ulong size = 1000;
```

typedef는 새로운 실제 자료형을 만드는 것이 아니라 기존 형에 별칭(alias)을 부여합니다.

typedef struct {...} Student; 형태를 사용하면 이후 struct 키워드를 반복하지 않아도 되어 코드가 간결해집니다.

5. 구조체 배열과 포인터

```
struct Student list[3];  
struct Student *p = list;  
p[0].id = 100;  
(p+1)->id = 200;
```

배열과 포인터 문법을 함께 사용하면 자료 관리 프로그램을 만들기 쉬워집니다. 연결 리스트 노드도 결국 "데이터 + 다음 노드를 가리키는 포인터"라는 구조체입니다.

C언어 ⑧ - 연결 리스트 완전 정리

단순·이중·환형 리스트, 삽입·삭제·탐색의 논리까지

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 왜 연결 리스트를 쓰는가

배열은 원소들이 연속된 공간에 있어 인덱스로 빠르게 접근하기 쉽지만, 중간 삽입/삭제 때 여러 원소를 이동해야 할 수 있습니다. 연결 리스트는 노드마다 다음 노드의 주소를 저장해 물리적으로 흩어진 노드를 연결합니다.

구조	핵심
단순 연결 리스트	각 노드가 next 하나를 가짐
이중 연결 리스트	prev와 next를 함께 가짐
환형 연결 리스트	마지막 노드가 다시 첫 노드로 연결

2. 노드 구조

```
typedef struct ListNode {
    int item;
    struct ListNode *next;
} ListNode;
```

핵심은 데이터와 다음 노드의 주소를 한 객체에 함께 저장하는 것입니다. head는 첫 번째 노드를 가리키는 포인터이고, 빈 리스트라면 head == NULL입니다.

3. 순회

```
ListNode *cur = ll->head;
while (cur != NULL) {
    printf("%d ", cur->item);
    cur = cur->next;
}
```

cur을 현재 노드로 두고 cur->next로 이동합니다. NULL을 만나면 끝입니다. 연결 리스트 문제에서 while(cur != NULL)은 매우 중요한 기본 패턴입니다.

4. 정렬 삽입

```
ListNode *cur = ll->head;
ListNode *prev = NULL;
while (cur != NULL && cur->item < item) {
    prev = cur;
    cur = cur->next;
}
```

C언어 ⑨ - 큐와 환형 큐

FIFO, front/rear, 배열 큐와 환형 큐의 동작 원리

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 큐란?

Queue는 먼저 들어온 데이터가 먼저 나가는 FIFO(First In, First Out) 구조입니다. 줄을 서는 것과 같은 개념입니다.

용어	의미
enqueue	데이터를 큐에 넣기
dequeue	데이터를 큐에서 빼기
front	가장 먼저 나갈 데이터 위치
rear	마지막에 들어온 데이터 위치/삽입 위치

2. 기본 배열 큐

```
int q[100];
int front=0, rear=0;

q[rear++] = item;    // enqueue
item = q[front++];   // dequeue
```

간단한 구현에서는 front와 rear를 증가시키면서 사용할 수 있습니다. 하지만 rear가 배열 끝에 도달하면 앞쪽에 비어 있는 공간이 있어도 못 쓰는 문제가 생길 수 있습니다.

3. 환형 큐

환형 큐는 배열의 끝 다음을 다시 배열의 시작으로 연결해 빈 공간을 재사용합니다. 인덱스를 $(index + 1) \% capacity$ 처럼 순환시킵니다.

```
rear = (rear + 1) % CAPACITY;
front = (front + 1) % CAPACITY;
```

환형 큐에서는 "가득 참"과 "비어 있음"을 구분하는 설계가 중요합니다. 한 칸을 비워 두는 방식, count를 따로 두는 방식 등 여러 구현이 있습니다.

4. 큐와 연결 리스트

연결 리스트로 큐를 만들면 배열 크기 제한을 덜 받습니다. head를 front, tail을 rear처럼 사용하면 enqueue와 dequeue를 $O(1)$ 에 가깝게 설계할 수 있습니다.

4. 여러 소스 파일

```
gcc main.c list.c -o app
```

프로그램이 커지면 기능을 여러 .c 파일로 나누고 헤더 .h에 선언을 모읍니다. 각 .c는 개별적으로 컴파일한 뒤 링크하는 식으로 확장할 수 있습니다.

5. C 표준 라이브러리

헤더	주요 기능
stdio.h	입출력: printf, scanf, fopen 등
stdlib.h	동적 메모리, 변환, 프로그램 종료 등
string.h	문자열 처리
math.h	수학 함수
ctype.h	문자 판별/변환
stdbool.h	bool, true, false(C99)
stdint.h	크기가 명시된 정수형
time.h	시간 관련 기능

헤더를 include했다고 모든 함수가 같은 식으로 동작하는 것은 아닙니다. 함수의 매개변수와 반환형, 필요한 라이브러리 링크 여부까지 확인해야 합니다.

6. 디버깅 기본

오류를 만나면 ① 컴파일 경고를 모두 확인하고 ② 변수의 값/주소를 출력하거나 디버거로 확인하고 ③ 포인터/배열 범위를 점검하고 ④ 한 번에 한 가지 가설만 검증하는 방식이 효율적입니다.

특히 "컴파일은 되는데 이상하게 실행된다"면 배열 범위 초과, 초기화되지 않은 변수, 잘못된 포인터, 메모리 해제 후 사용(use-after-free), 문자열 종료 문자 문제를 우선 의심하세요.

C언어 ⑪ - 표준 키워드 전체 지도

외우기보다 역할별로 분류해서 이해하기

학습 목표

이 문서는 업로드된 자료에서 빠진 설명까지 보완해, 단순 암기보다 "왜 그렇게 동작하는지"를 이해하는 데 초점을 맞췄습니다. 각 항목은 개념 → 문법 → 예제 → 자주 하는 실수 순으로 정리했습니다.

1. 자료형/수식 관련

분류	대표 키워드	핵심
기본형	char, int, float, double, void	기본 데이터 표현
변형	short, long, signed, unsigned	크기/부호 특성 조정
사용자 정의	struct, union, enum, typedef	새로운 데이터 표현 방법

2. 제어 관련

종류	키워드
조건/분기	if, else, switch, case, default
반복	for, while, do
반복 제어	break, continue
함수 종료	return

3. 저장 기간/링크 관련

키워드	역할
auto	지역 객체의 기본 저장 클래스
static	정적 저장 기간 또는 내부 링크 등 문맥에 따라 의미
extern	외부 정의 객체/함수에 대한 선언
register	레지스터 사용 힌트

4. 그 밖의 핵심

키워드	용도
const	수정하지 않을 값을 표현
volatile	외부 요인에 의해 바뀔 수 있음을 알리는 한정자
restrict	특정 포인터 기반 최적화 약속(C99)
inline	인라인화를 요청하는 함수 지정자
_Bool	C의 불리언 기본형

const는 "값이 절대 변경되지 않는다"는 시스템 전체 보장이 아니라 해당 이름을 통한 수정을 금지하는 타입 한정자라는 관점으로 이해하는 것이 정확합니다.

5. 외울 때의 전략

키워드만 가나다순으로 외우기보다 "자료형 → 제어 → 함수 → 저장 기간 → 사용자 정의 → 형 한정" 순으로 역할을 묶어 기억하세요. 코드를 읽다가 키워드를 만나면 "무엇을 바꾸는 키워드인가?"를 바로 말할 수 있는 것이 목표입니다.

```
printf("%d", **pp); // 10
```

pp는 p의 주소를 저장한다. 따라서 *pp는 p가 되고, **pp는 다시 value가 된다. 연결 리스트에서 head 포인터 자체를 함수에서 바꾸어야 할 때 이 개념이 등장할 수 있다.

12-2. 포인터와 배열 인덱스

```
int a[4] = {10, 20, 30, 40};
int *p = a;

printf("%d %d\n", a[2], *(p+2));
```

두 결과는 같다. a[i]는 포인터 관점에서 *(a+i)로 해석할 수 있다. 이 성질 때문에 배열과 포인터가 함께 설명되지만, 앞서 말했듯 배열과 포인터 변수 자체는 같은 객체가 아니다.

12-3. NULL, 잘못된 포인터, 범위

- NULL 포인터를 역참조하면 안 된다.
- 지역변수가 사라진 뒤 그 주소를 가리키는 포인터는 사용하면 안 된다.
- free한 메모리를 다시 사용하면 안 된다.
- 배열의 유효 범위를 벗어난 포인터/인덱스 접근은 정의되지 않은 동작의 원인이 된다.

안전한 습관
포인터를 만들었으면 “무엇의 주소를 가리키는지”, “그 대상의 수명이 언제 끝나는지”, “이 포인터를 지금 역참조해도 되는지”를 함께 확인한다.

13. C와 C++가 섞여 있는 자료 구분

업로드 자료 중 일부는 C++의 문법을 설명한다. C 공부를 하는 상황에서는 아래를 따로 떼어 놓아야 한다.

내용	C	C++
자료형 별칭	typedef가 대표적	using도 가능
참조(reference)	없음	int &ref = x; 가능
포인터	int *p, *p 역참조	포인터 사용 가능
멤버 접근	포인터는 ->	포인터는 ->
함수 오버로딩	지원하지 않음	지원
class	없음	지원

13-1. 특히 주의: using과 reference

```
// C++ 예시
using salary = double;
double x = 3.5;
double &ref = x;
```

이 문법은 C 코드에 그대로 넣으면 안 된다. C에서 자료형 별칭을 만들려면 typedef를 사용한다.

```
// C 예시
typedef double salary;
salary x = 3.5;
```

구분법
자료 이미지에 “C++”라고 적혀 있거나 using, reference(&) 같은 표현이 등장한다면 C 시험 범위와 분리해서 표시해두자.

14. 자주 틀리는 코드 패턴

14-1. scanf 의 주소 연산자

```
int n;
scanf("%d", &n);
```

scanf 는 입력받은 값을 변수에 써야 하므로 일반적인 변수 입력에서는 그 변수의 주소가 필요하다. 따라서 &n 을 전달한다. 반대로 printf 는 값을 읽어 출력하므로 n 자체를 전달한다.

함수	예	이유
scanf	scanf("%d", &n);	n에 쓸 주소 필요
printf	printf("%d", n);	n의 값을 읽어서 출력

14-2. ==와 =

```
if (a == 10) { ... } // 비교
a = 10; // 대입
```

14-3. 문자와 문자열

```
char c = 'A';
char s[] = "ABC";
```

14-4. sizeof 의 함정

```
int arr[5];
int *p = arr;
printf("%zu", sizeof(arr)); // 배열 전체
printf("%zu", sizeof(p)); // 포인터 크기
```

14-5. malloc 과 free

```
int *p = malloc(10 * sizeof *p);
if (!p) {
    return 1;
}
// 사용
free(p);
p = NULL;
```

15. 시험용 압축 암기표

15-1. 자료형

기억할 것	핵심
'A'	char/문자 하나
"ABC"	문자열
10	int 계열 정수 리터럴
3.14	double 리터럴
3.14f	float
100L	long
100LL	long long
unsigned	음수 없이 양의 범위 확대
sizeof(x)	바이트 크기

15-2. 포인터

표현	한 줄 해석
int *p	p 는 int 의 주소를 저장
&x	x 의 주소
*p	p 가 가리키는 대상의 값
p+1	가리키는 타입 한 칸만큼 이동
int **pp	포인터의 주소를 저장

15-3. 연산자

기호	핵심
=	대입
==	같은지 비교
!=	다른지 비교
&&	AND
	OR
!	NOT
%	나머지
++i	먼저 증가 후 사용
i++	먼저 사용 후 증가

15-4. 제어문

문법	핵심 질문
if	조건이 참이면 무엇을 실행?
switch	값에 따라 어느 case?
for	몇 번 반복?
while	조건이 참인 동안?
do-while	최소 한 번은 실행?
break	가장 가까운 반복/switch 종료
continue	현재 반복을 건너뛰고 다음 반복

01

선언 / 정의 / 초기화 / 대입

비슷해 보이지만 역할이 다른 네 개를 분리하면 C 코드가 훨씬 잘 읽힌다.

1. 선언(declaration)

"이 이름과 타입을 이렇게 사용할 수 있다"고 컴파일러에게 알리는 것이다. 함수 원형이나 extern 선언도 선언에 해당한다.

```
int x;  
int add(int a, int b);
```

2. 정의(definition)

실제로 객체를 만들거나 함수의 몸체를 제공하는 것. 일반적인 정의는 선언 역할도 함께 한다.

```
int x = 10;  
int add(int a, int b) { return a + b; }
```

3. 초기화(initialization)

객체가 처음 만들어질 때 값을 넣는 것. 대입과 다르다.

```
int x = 10; // 초기화  
x = 20;    // 대입
```

4. 대입(assignment)

이미 존재하는 객체에 새 값을 넣는다. 오른쪽 식을 계산한 뒤 왼쪽 대상에 결과를 저장한다.

핵심 기억법

"처음 만들면서 값 넣기 = 초기화", "만들어진 뒤 값 바꾸기 = 대입"

왜 중요한가? 전역/지역 변수, static 객체, const 객체, 배열 등의 초기화 규칙을 이해할 때 이 구분이 계속 등장한다.

02

값 범주와 형 변환

C에서는 서로 다른 타입의 값이 섞일 때 자동 변환이 일어나므로, 결과를 예측하려면 변환 규칙을 이해해야 한다.

1. 정수 승격(integer promotions)

작은 정수형(char, signed/unsigned char, short 등)은 연산 과정에서 int 또는 unsigned int로 승격될 수 있다. 따라서 "char끼리 더했는데 결과가 int처럼 보이는" 현상이 자연스럽다.

```
char a = 100;
char b = 20;
int c = a + b; // 보통 int 연산
```

2. 산술 변환의 대표적인 흐름

서로 다른 타입을 연산하면 더 넓은 표현이 가능한 타입 쪽으로 변환된다. 정확한 규칙은 복잡하지만, 실전에서는 "정수끼리인지, 부호가 섞였는지, 실수와 섞였는지"를 먼저 본다.

상황	예	관찰 포인트
정수 / 정수	5 / 2	결과는 정수 2
실수 / 정수	5.0 / 2	정수가 실수 쪽으로 변환
명시적 캐스팅	(double)5 / 2	5를 double로 바꾼 뒤 연산
부호 혼합	int + unsigned int	예상과 다른 부호/범위에 주의

3. 캐스팅(cast)

```
double avg = (double)sum / count;
```

캐스팅은 표현식을 원하는 타입으로 명시적으로 변환한다. 다만 "타입을 바꿨으니 데이터가 보존된다"는 뜻은 아니다. narrowing(좁아지는 변환)에서는 정보가 손실될 수 있다.

```
int x = 300;
unsigned char c = (unsigned char)x; // 범위 밖 값: 변환 결과를 확인해야 함
```

시험/실전 함정

(double)(a / b) 와 (double)a / b 는 다르다. 앞은 정수 나눗셈 후 실수화, 뒤는 실수 나눗셈이다.

```
int a = 5, b = 2;
printf("%.1f\n", (double)(a / b)); // 2.0
printf("%.1f\n", (double)a / b); // 2.5
```

03

const / static / extern 깊이 이해

이미 기억 클래스나 키워드를 배웠다면, 이번 장에서는 "이름·범위·수명·링크지"를 분리해서 본다.

1. const는 "수정하지 않겠다"는 제약을 표현한다.

```
const int MAX = 100;
// MAX = 200; // 컴파일 오류

void print(const char *s) {
    printf("%s\n", s);
}
```

const char *s는 "s가 가리키는 문자 데이터를 함수 안에서 수정하지 않겠다"는 뜻이다. 포인터 자체를 못 바꾸게 하는 것과는 다르다.

```
const int *p = &x; // *p 변경 금지, p 변경 가능
int * const p2 = &x; // p2 변경 금지, *p2 변경 가능
const int * const p3 = &x; // 둘 다 변경 금지
```

2. static의 핵심은 문맥에 따라 두 가지로 이해한다.

위치	효과	핵심 질문
함수 내부 static	호출이 끝나도 값이 유지됨	"지역인데 왜 값이 안 사라지지?"
파일 범위 static	그 파일 안에서만 이름을 사용	"다른 .c 파일에서 접근 가능한가?"

```
void counter(void) {
    static int n = 0;
    n++;
    printf("%d\n", n);
}
```

3. extern

다른 곳에 정의된 전역 객체/함수를 여기서 사용하겠다고 선언할 때 쓴다.

```
// a.c
int global = 10;

// b.c
extern int global;
int main(void) { printf("%d", global); }
```

한 단계 더

scope(어디서 이름을 볼 수 있는가), storage duration(객체가 얼마나 오래 존재하는가), linkage(서로 다른 선언이 같은 대상을 가리키는가)는 서로 다른 축이다.

04

메모리의 실제 모습

포인터를 제대로 이해하려면 "변수 하나 = 메모리 한 칸"이라는 그림을 넘어 프로그램 메모리의 큰 구조를 알아야 한다.

영역	대표 내용	일반적 특징
Text / 코드	기계어 명령	실행 코드가 위치
Read-only data	문자열 리터럴 등	수정하면 안 되는 데이터가 포함될 수 있음
Data/BSS	전역/정적 객체	프로그램 시작부터 종료까지 유지
Stack	지역 변수, 호출 프레임	함수 호출과 함께 생성/정리
Heap	malloc/calloc 등 동적 할당	직접 할당/해제 책임

이 구분은 정확한 구현 세부와는 차이가 있을 수 있지만, 코드에서 "왜 이 포인터가 오래 살아남는가?", "왜 지역 변수 주소를 반환하면 위험한가?"를 이해하는 데 매우 유용하다.

```
int *bad(void) {
    int local = 10;
    return &local; // 반환 후 local의 수명이 끝남 -> 잘못된 포인터
}
```

함수에서 지역 변수의 주소를 반환하는 것은 대표적인 dangling pointer 문제다. 반면 static 지역 변수는 함수가 끝나도 객체의 수명이 유지된다.

```
int *okay(void) {
    static int value = 10;
    return &value;
}
```

중요

"주소가 있다"와 "그 주소를 계속 사용해도 된다"는 같은 말이 아니다. 포인터의 유효성은 대상 객체의 수명과 함께 판단한다.

05

동적 메모리 안전 규칙

malloc 자체보다 더 중요한 것은 "누가 소유하고 언제 해제하는가"다.

1. malloc / calloc / realloc / free 의 역할

함수	무엇을 하는가	주의
malloc(n)	n 바이트 확보	초기화되지 않음
calloc(count,size)	count*size 확보 + 0 으로 초기화	곱셈 크기 계산 주의
realloc(p,n)	기존 블록 크기 변경 시도	이동될 수 있음
free(p)	할당한 블록 해제	해제 후 사용 금지

2. realloc 의 안전한 패턴

```
int *tmp = realloc(p, new_count * sizeof(*p));
if (tmp != NULL) {
    p = tmp;
    // 성공
} else {
    // p 는 아직 유효하므로 기존 메모리를 계속 처리 가능
}
```

왜 tmp 를 쓰나?

realloc 에 실패하면 원래 p 가 그대로 남아 있을 수 있다. 곧바로 p = realloc(p, ...)라고 쓰면 원래 주소를 잃을 위험이 있다.

3. 반드시 구별해야 할 네 가지

- memory leak: 할당한 메모리를 잃어버려 free 할 수 없는 상태
- dangling pointer: 대상이 사라졌는데 포인터만 남아 있는 상태
- double free: 같은 메모리를 두 번 해제
- use-after-free: free 한 뒤 그 주소를 다시 읽거나 씀

```
free(p);
p = NULL; // 같은 포인터를 다시 실수로 free 하는 상황을 줄이는 습관
```

free(NULL)는 아무 작업도 하지 않으므로, "사용 후 NULL 로 만들기"는 실수 방지에 도움이 된다.

06

함수 포인터와 콜백

C가 "함수를 값처럼 전달"하는 핵심 개념이다. 정렬 함수, 이벤트 처리, 자료구조의 비교 함수에서 특히 중요하다.

1. 함수 포인터 읽는 법

```
int add(int a, int b) { return a + b; }
int (*fp)(int, int) = add;

printf("%d\n", fp(2, 3));
```

fp는 "int 두 개를 받아 int를 반환하는 함수의 주소"를 저장한다. 함수 이름 add는 식에서 함수 포인터처럼 변환될 수 있으므로 fp = add와 fp = &add 모두 사용할 수 있다.

2. 콜백(callback)

함수가 다른 함수에게 "나중에 호출할 함수"를 인자로 넘기는 방식이다.

```
int calc(int a, int b, int (*op)(int, int)) {
    return op(a, b);
}

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

printf("%d\n", calc(10, 3, add));
printf("%d\n", calc(10, 3, sub));
```

3. qsort와 연결되는 사고방식

표준 라이브러리 정렬은 "어떤 값을 어떤 순서로 비교할지"를 호출자가 함수로 알려준다. 따라서 함수 포인터를 이해하면 라이브러리 함수의 설계가 훨씬 자연스럽게 보인다.

07

재귀와 호출 스택

재귀는 반복문보다 특별한 문법이 아니라, 함수가 자기 자신을 다시 호출하는 제어 방식이다.

재귀 함수에는 거의 항상 두 요소가 필요하다.

- 기저 조건(base case): 더 이상 호출하지 않고 끝나는 조건
- 진행 규칙: 현재 문제를 더 작은 문제로 바꾸어 재귀 호출

```
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

factorial(4)는 $4 * \text{factorial}(3)$, 다시 $3 * \text{factorial}(2)$... 식으로 쌓인다. 각 호출은 자신의 매개변수와 지역 변수, 반환 위치를 가진 호출 프레임을 사용한다.

재귀를 볼 때

① 종료 조건은? ② 한 번 호출될 때 입력이 작아지는가? ③ 최대 깊이는 얼마나 되는가? 이 세 가지를 먼저 본다.

재귀가 항상 좋은 것은 아니다. 단순 반복이 더 읽기 쉽고 메모리 사용도 예측하기 쉬운 경우가 많다. 트리 탐색처럼 문제 구조 자체가 재귀적일 때 특히 유용하다.

08

argc / argv

프로그램을 실행할 때 뒤에 적은 문자열을 C 프로그램이 어떻게 받는지 알아두면 파일명, 옵션, 간단한 도구를 만들기 쉬워진다.

```
int main(int argc, char *argv[]) {
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    return 0;
}
```

예를 들어 ./app input.txt 42 로 실행하면 argv[0]은 실행 파일 이름, argv[1]은 input.txt, argv[2]는 문자열 "42"가 된다. 숫자라고 자동으로 int 가 되는 것은 아니다.

```
long value = strtol(argv[2], NULL, 10);
```

문자열을 숫자로 바꿀 때는 atoi 보다 strtol 계열이 오류와 범위를 확인하기 좋다.

09

파일 입출력

파일은 "열기 -> 사용 -> 닫기"라는 자원 관리 순서로 생각하면 된다.

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");
    return 1;
}

char buf[100];
while (fgets(buf, sizeof buf, fp) != NULL) {
    printf("%s", buf);
}

fclose(fp);
```

모드	의미
r	읽기
w	쓰기 - 기존 내용이 지워질 수 있음
a	끝에 추가
rb / wb	바이너리 모드

1. EOF를 "문자 하나"라고 생각하지 않는다. EOF는 입력 함수가 더 읽을 수 없을 때 반환하는 특별한 상태를 나타내는 정수 값이다.

2. fgets는 줄 단위 읽기에 편리하며 버퍼 크기를 함께 전달한다. scanf("%s", ...)처럼 무제한 입력을 받는 습관은 피한다.

3. fscanf / fprintf, fread / fwrite, fseek / ftell 등은 목적에 따라 선택한다. 텍스트냐 바이너리냐부터 먼저 결정한다.

자원 관리 규칙

fopen 성공 -> 반드시 fclose. 파일을 못 열었다면 FILE*가 NULL 인지 먼저 확인한다.

10

오류 처리와 방어적 C

C에는 예외가 자동으로 모든 것을 처리해주는 시스템이 없다. 성공/실패를 호출자가 직접 검사하는 습관이 매우 중요하다.

1. 반환값 검사

```
if (scanf("%d", &x) != 1) {
    fprintf(stderr, "invalid input\n");
    return 1;
}
```

2. NULL 검사

```
int *p = malloc(sizeof *p);
if (p == NULL) {
    fprintf(stderr, "allocation failed\n");
    return 1;
}
```

3. errno와 perror

일부 표준 라이브러리 함수는 실패 상황에서 errno를 설정할 수 있다. perror는 현재 errno에 기반한 사람이 읽을 수 있는 오류 메시지를 출력한다.

```
FILE *fp = fopen("missing.txt", "r");
if (fp == NULL) {
    perror("fopen");
    return 1;
}
```

4. assert

```
#include <assert.h>
assert(ptr != NULL);
```

프로그램이 절대 성립해야 하는 내부 조건을 개발 단계에서 확인할 때 쓴다. 사용자 입력 검증용 일반 오류 처리와는 목적이 다르다.

방어적 C의 질문

"이 함수가 실패하면 무엇을 반환하지?", "포인터가 NULL 이면?", "입력이 범위를 넘으면?", "파일이 없으면?"를 코드 작성 단계에서 먼저 생각한다.

11

비트 마스킹 실전

비트 연산자를 배웠다면 이제 "플래그 여러 개를 하나의 정수에 저장"하는 패턴까지 연결해두는 것이 좋다.

```
#define READ  0x01
#define WRITE 0x02
#define EXEC  0x04

unsigned flags = READ | WRITE;
```

동작	표현	뜻
켜기	flags = WRITE	WRITE 비트 추가
끄기	flags &= ~WRITE	WRITE 비트 제거
검사	if (flags & WRITE)	WRITE 가 켜져 있는가?
토글	flags ^= WRITE	현재 상태 반전

각 플래그가 서로 다른 비트 하나를 차지하도록 만들면, 하나의 정수로 여러 boolean 상태를 관리할 수 있다. 네트워크/하드웨어/권한/상태 값에서 자주 쓰인다.

주의

비트 연산에서는 부호 있는 정수보다 unsigned 를 사용하면 의도가 명확하고 비트 이동 관련 혼란을 줄이기 쉽다.

12

헤더와 다중 파일 프로그램

프로그램이 커지면 한 .c 파일에 전부 넣는 방식이 유지보수에 불리하다. 선언과 구현을 분리하는 습관이 중요하다.

일반적인 구조

```
project/  
  main.c  
  list.c  
  list.h
```

```
// list.h  
#ifndef LIST_H  
#define LIST_H  
  
void list_init(void);  
int list_size(void);  
  
#endif
```

```
// list.c  
#include "list.h"  
  
void list_init(void) { /* 구현 */ }  
int list_size(void) { return 0; }
```

header guard는 같은 헤더가 여러 경로로 포함되어도 선언이 중복되는 문제를 막는 대표적인 패턴이다. 현대 환경에서는 #pragma once도 널리 쓰이지만, 표준 C만 놓고 보면 include guard가 기본적인 이식 가능한 방법이다.

여러 파일에서 같은 전역 변수를 직접 공유하기보다, 필요한 데이터를 함수나 구조체로 감싸고 인터페이스(.h)를 통해 접근하도록 만들면 결합도를 낮출 수 있다.

선언 vs 정의 재등장

.h에는 보통 "무엇이 있는가"를 알리는 선언, .c에는 "실제로 어떻게 동작하는가"를 구현한다. 단, 상황에 따라 헤더에 정의가 들어가는 경우도 있으므로 무조건적인 규칙으로 외우지는 않는다.

13

컴파일러 경고와 디버깅

코드가 실행되기 전 잡을 수 있는 오류를 최대한 많이 잡는 것이 가장 싹 디버깅이다.

GCC에서 자주 쓰는 옵션

```
gcc -std=c17 -Wall -Wextra -g main.c list.c -o app
```

옵션	역할
-Wall	대표적인 경고 활성화
-Wextra	추가 경고 활성화
-g	디버거용 정보 포함
-std=c17	사용할 C 표준 지정

디버거에서 중요한 개념: breakpoint(중단점), step/next(한 단계 실행), print(값 확인), backtrace(호출 스택), watch(값 변화 감시).

```
gdb ./app
(gdb) break main
(gdb) run
(gdb) next
(gdb) print x
(gdb) backtrace
```

메모리 오류에는 sanitizer가 특히 유용하다.

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined main.c -o app
```

AddressSanitizer는 use-after-free, buffer overflow 같은 문제를 찾는 데, UndefinedBehaviorSanitizer는 여러 종류의 정의되지 않은 동작을 탐지하는 데 도움을 준다.

습관 하나만 고른다면

경고 0개를 목표로 컴파일한다. "경고는 그냥 무시해도 된다"는 습관이 나중에 포인터/메모리 오류로 이어질 수 있다.

보충 삽입 C - GDB watch 실제 사용 예

이번 대조에서 추가된 누락 내용

기존 통합본에는 watch가 "값 변화 감시"라는 개념으로 들어 있지만, 실제 명령 흐름을 한 번에 볼 수 있는 예제가 빠져 있었다.

```
gdb ./app
(gdb) break main
(gdb) run
(gdb) watch some_variable
(gdb) next
(gdb) print x
(gdb) backtrace
```

`watch some\_variable`은 해당 변수의 값이 바뀌는 순간 프로그램을 멈추게 하여, 언제 어디서 값이 변경됐는지 찾을 때 유용하다.

14

Undefined Behavior 와 흔한 사고

C 에서 가장 무서운 문장은 "컴파일은 되는데 가끔 이상하게 작동한다"이다. 그 배경에 정의되지 않은 동작이 있을 수 있다.

사고	문제
배열 범위 밖 접근	무엇이 나올지 보장되지 않음
해제 후 사용	dangling/use-after-free
초기화하지 않은 자동 변수 읽기	유효한 값을 보장하지 못함
NULL 역참조	실행 중 오류 가능
부호 있는 정수 오버플로	정의되지 않은 동작
잘못된 포인터 역참조/정렬 위반	유효하지 않은 메모리 접근 가능

특히 배열 범위는 "0 부터 시작"이라는 기본 원칙 때문에 off-by-one 실수가 많다.

```
int a[5];
for (int i = 0; i < 5; i++)
    a[i] = i;      // OK

// a[5] = 10;      // 범위 밖
```

문자열은 마지막에 `\0` 이 필요하다는 것도 같은 맥락이다.

```
char s[4] = "abc";    // a b c \0 -> OK
char t[3] = "abc";    // 문자열용으로는 끝의 \0 공간이 없음
```

핵심 태도

"내 컴퓨터에서 지금 잘 돌아간다"는 정답 증명이 아니다. C에서는 언어가 보장하는 범위 안에서 코드가 동작하는지를 봐야 한다.

15

검색·정렬의 표준 도구

자료구조를 직접 구현하는 것과 라이브러리를 사용하는 것은 서로 다른 능력이다. 표준 함수의 인터페이스를 읽는 연습도 중요하다.

1. qsort

```
#include <stdlib.h>

int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int a[] = {4, 1, 3, 2};
qsort(a, 4, sizeof a[0], cmp_int);
```

비교 함수의 인자는 void*다. 따라서 실제 타입으로 캐스팅한 뒤 역참조한다. 비교 함수에서 단순히 x-y를 반환하는 코드는 큰 정수에서 오버플로 위험이 있으므로 비교식 형태가 더 안전하다.

2. bsearch

```
int key = 3;
int *found = bsearch(&key, a, 4, sizeof a[0], cmp_int);
if (found != NULL) printf("%d\n", *found);
```

bsearch는 정렬된 범위에서 사용하는 이진 탐색이다. "정렬되어 있지 않는데 bsearch를 호출"하면 원하는 결과를 기대할 수 없다.

16

문제 풀이용 실전 패턴

문법을 많이 아는 것과 문제를 잘 푸는 것은 다르다. 반복해서 등장하는 사고 패턴을 익혀두면 코드가 빨라진다.

패턴	핵심 변수	대표 문제
카운팅	count	조건을 만족하는 개수
누적	sum	합/평균/가중합
최대값/최소값	best	범위의 극값
빈도 배열	freq[]	값의 출현 횟수
두 포인터	left/right	정렬 배열, 구간 탐색
상태 변수	state	입력 흐름/모드 관리
방문 배열	visited[]	중복 방문 방지

```
int count = 0;
for (int i = 0; i < n; i++)
    if (a[i] % 2 == 0)
        count++;
```

이런 단순한 패턴을 먼저 코드로 만들고, 그 안에 필요한 조건을 넣는 방식이 안정적이다. 한 번에 복잡한 코드를 쓰기보다 "입력 -> 상태 변화 -> 출력"을 분리해서 생각한다.

연결 리스트에도 그대로 적용

cur, prev, next 같은 이름은 임의의 변수처럼 보이지만 사실 "현재 노드 / 이전 노드 / 다음 노드"라는 상태를 명시하는 문제 해결 패턴이다. 포인터 코드를 읽을 때 변수의 역할을 먼저 적어보면 훨씬 쉽다.

초압축 레퍼런스

기호/표현	먼저 떠올릴 것
&x	x의 주소
*p	p가 가리키는 대상
p + 1	가리키는 타입 기준으로 한 원소 이동
arr[i]	*(arr + i)와 같은 의미
const int *p	*p 수정 금지
int * const p	p 자체 변경 금지
sizeof arr	배열 자체라면 전체 바이트 수
sizeof ptr	포인터 변수 자체의 크기
malloc	초기화 안 된 동적 메모리
calloc	0으로 초기화된 동적 메모리
realloc	주소가 바뀔 수 있음
free	동적 메모리 해제
FILE *	파일 스트림 핸들
argc/argv	명령행 인자
qsort	함수 포인터를 이용한 범용 정렬
extern	외부에 정의된 대상 사용 선언
static local	호출 사이에도 값 유지
static file-scope	외부 파일에서 이름 숨김
-Wall -Wextra	컴파일 경고 강화
-g	디버깅 정보
-fsanitize=address,undefined	메모리/UB 탐지에 도움

이 페이지를 외워두면 앞에서 만든 자료형/연산자/제어문/배열/포인터/함수/구조체/연결리스트/큐 자료와 연결하기 쉽다.

01. 문제 풀이 전 C 핵심 복습

자료구조 코드는 새로운 문법을 사용하는 것처럼 보이지만, 실제로는 기존 C 문법을 여러 개 묶어서 사용하는 것입니다. 특히 포인터, 구조체, 함수, 동적 메모리가 동시에 등장합니다.

1-1. 반드시 익숙해야 하는 문법

- 변수는 값을 저장하고, 포인터는 다른 변수/노드의 주소를 저장한다.
- `&x`는 x의 주소, `*p`는 p가 가리키는 곳의 실제 값을 의미한다.
- 구조체 변수는 `.`, 구조체 포인터는 `->` 로 멤버에 접근한다.
- NULL은 “유효한 대상이 없다”는 의미로 포인터 비교에 사용한다.
- malloc은 동적 메모리를 만들고, free는 그 메모리를 반환한다.
- 함수는 값을 복사해서 받을 수도 있고, 포인터를 받아 원본을 바꿀 수도 있다.

```
int x = 10;
int *p = &amp;x;

printf("%d\n", x); // 10
printf("%p\n", (void*)p); // x의 주소
printf("%d\n", *p); // 10

*p = 20; // x의 값이 20으로 바뀜
```

1-2. 자료구조에서 &와 *가 계속 나오는 이유

노드 하나를 만들고 그 주소를 저장하거나, 함수에서 head를 바꿔야 하기 때문입니다. 예를 들어 list의 head가 첫 노드를 가리키는데, 첫 노드를 삭제하면 head 자체가 다른 노드를 가리키도록 바뀌어야 합니다. 이때 함수에 head의 주소를 넘기는 방식이 사용될 수 있습니다.

```
void changeHead(Node **head) {
    *head = (*head)->next;
}

// Node *head가 아니라 Node **head를 쓰는 이유:
// 함수가 호출한 쪽의 head 변수 자체를 바꿔야 하기 때문.
```

1-3. 배열과 포인터의 관계

표현	의미
a	배열 이름은 대부분의 식에서 첫 원소의 주소처럼 사용됨
&a[0]	첫 원소의 주소
a[i]	i번째 원소의 값
*(a+i)	a[i]와 같은 의미
&a[i]	i번째 원소의 주소

02. 포인터와 구조체: 자료구조의 언어

Linked List와 Tree에서 가장 많이 보게 되는 형태가 “자기와 같은 종류를 가리키는 포인터”입니다. 이것이 노드 연결의 기본입니다.

2-1. 가장 기본적인 Node

```
typedef struct ListNode {
    int item;
    struct ListNode *next;
} ListNode;
```

```
ListNode *head = NULL;
```

여기서 핵심은 ListNode 안에 ListNode 자체가 통째로 들어가는 것이 아니라, ListNode를 가리키는 포인터만 들어간다는 점입니다. 포인터의 크기는 고정되어 있으므로 이런 자기 참조 구조체를 만들 수 있습니다.

2-2. 구조체 포인터 접근

```
ListNode node;
node.item = 10;
```

```
ListNode *p = &node;
p->item = 20;
```

// p->item 은 (*p).item 과 같은 뜻

2-3. double pointer가 필요한 대표 상황

상황	예
head 자체 변경	첫 노드 삭제, 빈 리스트에 첫 노드 삽입
root 자체 변경	BST에서 root 삭제 후 새 root가 필요할 때
함수에서 포인터를 생성하고 반환 대신 수정	Node **로 주소를 전달
주의	단순히 노드의 내용만 수정할 때는 Node *로 충분한 경우가 많음

2-4. 포인터를 읽는 순서

예: Node **pp일 때 *pp는 Node *이고, **pp는 Node입니다. 괄호를 직접 써보면 훨씬 덜 헷갈립니다. 자료구조 코드에서 (*cur)->next 같은 표현이 나오는 이유도 “무엇이 포인터인가”를 정확히 구분하기 위해서입니다.

```
Node *a; // 노드를 가리킴
Node **b; // Node *를 가리킴
```

```
*b // Node *
**b // Node
```

03. 동적 메모리와 수명 관리

자료구조는 실행 중 필요한 만큼 노드를 만들고 없애야 하므로 malloc/free가 매우 중요합니다.

3-1. malloc의 기본

```
#include <stdlib.h>

Node *newNode = malloc(sizeof(Node));
if (newNode == NULL) {
    // 메모리 할당 실패 처리
}
newNode->item = 10;
newNode->next = NULL;
```

3-2. sizeof는 “개수”가 아니라 “바이트 크기”

sizeof(Node)는 Node 하나를 저장하는 데 필요한 바이트 수입니다. 배열을 동적으로 만들 때는 sizeof(Node) * count처럼 사용합니다. 자료구조에서는 malloc(sizeof(Node))처럼 작성하면 타입을 직접 세는 실수를 줄일 수 있습니다.

3-3. free 이후에 하면 안 되는 것

```
free(cur);

// cur을 다시 사용하면 안 됨.
// 특히 cur->next를 free 뒤에 읽는 것은 위험.

// 안전한 패턴
Node *next = cur->next;
free(cur);
cur = next;
```

3-4. 대표 메모리 오류

오류	의미	예
Memory leak	할당한 메모리를 free하지 못함	삭제한 노드의 free 누락
Dangling pointer	이미 free한 곳을 가리키는 포인터	free(p) 후 p->item 접근
Double free	같은 메모리를 두 번 free	free(p); free(p);
NULL dereference	NULL을 역참조	p == NULL인데 p->next
Out-of-bounds	배열 범위를 넘음	a[n] 접근

3-5. 자료구조 문제에서 가장 중요한 수명 규칙

“삭제 대상의 연결 정보를 먼저 확보하고, 연결을 바꾼 뒤, 마지막에 free한다.” 이 순서를 익히면 많은 삭제 문제를 해결할 수 있습니다.

04. Linked List

연결 리스트의 본질은 “노드가 다음 노드의 주소를 가지고 있다”는 것입니다. 배열처럼 연속된 메모리에 있을 필요가 없고, 삽입/삭제 때 필요한 연결만 바꿉니다.

4-1. 단순 연결 리스트 그림

```
head
↓
[10 | next] -&gt; [20 | next] -&gt; [30 | NULL]
```

4-2. 순회(traversal)

```
Node *cur = head;
while (cur != NULL) {
    printf("%d ", cur-&gt;item);
    cur = cur-&gt;next;
}
```

포인트: cur은 head 자체가 아니라 “현재 보고 있는 노드”입니다. cur을 이동시켜도 head는 그대로입니다. 그래서 탐색에는 cur을 사용하는 것이 기본입니다.

4-3. 맨 앞 삽입

```
Node *n = malloc(sizeof(Node));
n-&gt;item = item;
n-&gt;next = head;
head = n;
```

순서가 중요합니다. n->next에 기존 head를 저장한 뒤 head를 n으로 옮겨야 기존 리스트가 끊기지 않습니다.

4-4. 중간 삽입: prev / cur

```
Node *prev = NULL;
Node *cur = head;

while (cur != NULL && cur-&gt;item < item) {
    prev = cur;
    cur = cur-&gt;next;
}

// 새 노드 n을 cur 앞에 삽입
n-&gt;next = cur;
if (prev == NULL)
    head = n;
else
    prev-&gt;next = n;
```

여기서 prev == NULL은 “삽입 위치가 맨 앞”이라는 뜻입니다. 연결 리스트 문제에서 첫 노드가 특별히 취급되는 이유가 바로 이것입니다.

4-5. 삭제

```
Node *prev = NULL;
Node *cur = head;

while (cur != NULL && cur-&gt;item != item) {
    prev = cur;
    cur = cur-&gt;next;
}

if (cur == NULL) {
    // 없음
} else if (prev == NULL) {
    head = cur-&gt;next;
    free(cur);
} else {
    prev-&gt;next = cur-&gt;next;
    free(cur);
}
```

4-6. 꼭 확인해야 하는 예외

상황	확인
빈 리스트	head == NULL
첫 노드 대상	prev == NULL
마지막 노드 대상	cur->next == NULL
하나뿐인 노드	head == cur && cur->next == NULL
없는 값	끝까지 갔으므로 cur == NULL

4-7. 이중 연결 리스트

```
typedef struct Node {
    int item;
    struct Node *prev;
    struct Node *next;
} Node;
```

삭제할 때 앞뒤 노드를 직접 연결할 수 있다는 장점이 있지만, prev와 next 두 방향을 모두 정확히 갱신해야 합니다.

4-8. 환형 리스트

마지막 노드의 next가 NULL이 아니라 첫 노드(head)를 가리키는 구조입니다. 따라서 순회 종료 조건을 NULL로 잡으면 무한 루프가 생깁니다. 특정 노드로 다시 돌아왔는지 비교하는 방식 등을 사용합니다.

05. Stack

Stack은 LIFO(Last In, First Out), 즉 “마지막에 들어간 것이 먼저 나온다”가 핵심입니다. 접시를 위에 쌓고 위에서부터 꺼내는 모습으로 생각하면 됩니다.

5-1. 핵심 연산

연산	의미
push	데이터 넣기
pop	맨 위 데이터 제거 + 가져오기
peek/top	맨 위 데이터만 확인
isEmpty	비어 있는지 확인
isFull	배열 기반에서 가득 찼는지 확인

5-2. 배열 기반 Stack

```
#define MAX 100
int stack[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) return;
    stack[++top] = x;
}

int pop(void) {
    if (top == -1) return -1;
    return stack[top--];
}
```

top = -1이면 비어 있음. push할 때 ++top 후 저장하고, pop할 때 현재 값을 꺼낸 뒤 top--하는 패턴이 자주 등장합니다.

5-3. 연결 리스트 기반 Stack

head를 top처럼 사용하면 맨 앞 삽입/삭제가 O(1)입니다. push는 새 노드를 head 앞에 붙이고, pop은 head를 다음 노드로 이동시킨 뒤 기존 head를 free합니다.

```
void push(Node **top, int x) {
    Node *n = malloc(sizeof(Node));
    n->item = x;
    n->next = *top;
    *top = n;
}
```

5-4. Stack에서 자주 나오는 응용

괄호 검사, 후위 표기식, 재귀 호출의 실행 흐름 이해, DFS(깊이 우선 탐색)의 보조 구조 등과 연결됩니다. 따라서 Stack은 “자료구조 하나”라기보다 실행 순서를 되돌리거나 마지막 작업부터 처리해야 할 때 쓰는 사고방식입니다.

06. Queue / Circular Queue

Queue는 FIFO(First In, First Out), 즉 “먼저 들어온 것이 먼저 나온다”입니다. 은행 줄처럼 생각하면 됩니다.

6-1. 핵심 포인터

변수	역할
front	가장 먼저 꺼낼 위치
rear	마지막으로 넣은 위치 또는 다음 삽입 위치(구현에 따라 다름)
size/count	현재 원소 개수(선택 사항)

6-2. 배열 기반 선형 Queue의 문제

앞에서 dequeue를 반복하면 배열 앞부분에 빈 공간이 생깁니다. rear가 배열 끝에 도달하면 실제로는 빈 칸이 남아 있어도 더 넣지 못할 수 있습니다. 이 문제를 해결하는 대표 방법이 원형 큐입니다.

6-3. Circular Queue의 핵심

```
#define CAP 8
int q[CAP];
int front = 0;
int rear = 0;
int count = 0;

// enqueue
q[rear] = x;
rear = (rear + 1) % CAP;
count++;

// dequeue
x = q[front];
front = (front + 1) % CAP;
count--;
```

% CAP이 핵심입니다. rear가 CAP-1까지 갔다면 다음 위치가 0으로 돌아옵니다. count를 두면 empty(count==0)와 full(count==CAP)를 쉽게 구분할 수 있습니다.

6-4. Queue 구현에서 반드시 정해야 할 것

rear를 “마지막 원소”로 돌지, “다음 삽입 위치”로 돌지 먼저 정해야 합니다. 두 방식을 섞으면 off-by-one 오류가 매우 쉽게 발생합니다. 과제의 starter code가 있다면 그 정의를 먼저 따라야 합니다.

6-5. Stack과 Queue 비교

구조	삽입	삭제	핵심
Stack	push	pop	LIFO
Queue	enqueue	dequeue	FIFO
Circular Queue	enqueue	dequeue	배열 공간 재사용

07. Binary Tree

Binary Tree는 각 노드가 최대 두 개의 자식(left, right)을 갖는 트리입니다. Linked List에서 next 하나를 다루던 것에서 left/right 두 방향을 다룬다고 생각하면 접근이 쉽습니다.

7-1. 기본 용어

용어	뜻
root	트리의 시작 노드
parent	자식 노드를 가진 상위 노드
child	parent 아래의 노드
leaf	자식이 없는 노드
depth	root에서 얼마나 떨어졌는지
height	가장 깊은 leaf까지의 거리(정의에 따라 edge/node 기준 차이 가능)
subtree	어떤 노드를 root로 보는 부분 트리

7-2. Node 구조

```
typedef struct TreeNode {
    int key;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;
```

7-3. Tree는 재귀와 공합이 좋다

“현재 노드에서 할 일” + “왼쪽 서브트리에서 같은 일” + “오른쪽 서브트리에서 같은 일”이라는 구조가 반복되므로 재귀 함수가 자연스럽습니다. 단, 종료 조건으로 root == NULL을 정확히 작성해야 합니다.

7-4. 세 가지 순회

```
Preorder : root -> left -> right
Inorder  : left -> root -> right
Postorder: left -> right -> root
```

```
예시
    4
   / \
  2   6
 / \ / \
1 3 5 7
```

순회	방문 순서
Preorder	4 2 1 3 6 5 7
Inorder	1 2 3 4 5 6 7
Postorder	1 3 2 5 7 6 4

Inorder가 정렬된 순서가 되는 것은 “그 트리가 BST일 때”입니다. 일반 Binary Tree의 inorder가 항상 정렬되는 것은 아닙니다.

7-5. 순회 코드

```
void inorder(TreeNode *root) {
    if (root == NULL) return;
    inorder(root->left);
```

```
    printf("%d ", root->key);  
    inorder(root->right);  
}
```

08. Binary Search Tree (BST)

BST는 Binary Tree에 “순서 규칙”을 추가한 구조입니다. 일반적으로 어떤 노드 기준으로 왼쪽 서브트리의 값은 더 작고, 오른쪽 서브트리의 값은 더 큼니다. 중복값 정책은 과제에서 별도로 정할 수 있습니다.

8-1. 가장 중요한 규칙

```

      8
     / \
    3  12
   / \ / \
  1 5 10 14

```

현재 값 x와 비교

```

x < root->key -> left
x > root->key -> right
x == root->key -> 중복 처리 정책

```

8-2. 검색(search)

```

TreeNode *search(TreeNode *root, int key) {
    if (root == NULL) return NULL;
    if (key == root->key) return root;
    if (key < root->key)
        return search(root->left, key);
    return search(root->right, key);
}

```

8-3. 삽입(insert)

```

TreeNode *insert(TreeNode *root, int key) {
    if (root == NULL) {
        TreeNode *n = malloc(sizeof(TreeNode));
        n->key = key;
        n->left = n->right = NULL;
        return n;
    }

    if (key < root->key)
        root->left = insert(root->left, key);
    else if (key > root->key)
        root->right = insert(root->right, key);

    return root;
}

```

여기서 `root->left = insert(root->left, key)`처럼 “재귀 호출의 반환값을 다시 연결”하는 것이 아주 중요합니다. 왼쪽 서브트리가 NULL이었으면 재귀 함수가 새 노드 주소를 반환하고, 그것을 left에 저장해야 연결이 생깁니다.

8-4. 삭제가 가장 어려운 이유

BST의 삭제는 세 경우로 나뉘어 생각하면 됩니다. (1) leaf: 그냥 제거, (2) 자식 하나: 자식을 그 자리에 연결, (3) 자식 둘: 후계자(successor) 또는 전임자(predecessor)의 값을 가져온 뒤 그 노드를 삭제합니다.

8-5. 삭제 3가지 경우

경우	현재 노드의 상태	처리
1	<code>left == NULL && right == NULL</code>	NULL 반환
2	자식 하나	존재하는 자식 반환
3	자식 둘	오른쪽의 최소값(successor)을 복사하고 successor 삭제

8-6. successor 찾기

```

TreeNode *minValueNode(TreeNode *node) {

```

```

TreeNode *cur = node;
while (cur != NULL && cur->left != NULL)
    cur = cur->left;
return cur;
}

```

8-7. 삭제 구현

```

TreeNode *deleteNode(TreeNode *root, int key) {
    if (root == NULL) return NULL;

    if (key < root->key) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->key) {
        root->right = deleteNode(root->right, key);
    } else {
        if (root->left == NULL) {
            TreeNode *r = root->right;
            free(root);
            return r;
        }
        if (root->right == NULL) {
            TreeNode *l = root->left;
            free(root);
            return l;
        }

        TreeNode *s = minValueNode(root->right);
        root->key = s->key;
        root->right = deleteNode(root->right, s->key);
    }
    return root;
}

```

BST 삭제의 핵심은 “삭제한 뒤 그 자리에 무엇을 반환해서 연결할 것인가”입니다. return root, return child, return NULL을 그림으로 먼저 이해하면 코드가 훨씬 쉽게 읽힙니다.

09. 재귀와 순회 패턴

Linked List와 Tree 문제에서 재귀는 자주 등장합니다. 재귀는 어렵게 생각하지 말고 “기저 조건 + 현재 단계 + 더 작은 문제로 이동” 세 부분으로 분해하세요.

9-1. 재귀의 3요소

요소	예
Base case	root == NULL
Current work	현재 node 값 출력/비교
Recursive call	left 또는 right로 이동

9-2. List 재귀

```
void printList(Node *p) {
    if (p == NULL) return;
    printf("%d ", p->item);
    printList(p->next);
}
```

9-3. Tree 재귀

```
void preorder(TreeNode *root) {
    if (root == NULL) return;
    printf("%d ", root->key);
    preorder(root->left);
    preorder(root->right);
}
```

9-4. return을 연결로 사용하는 패턴

BST 삽입/삭제에서 가장 중요한 재귀 패턴입니다. 재귀 호출이 “수정된 서브트리의 새 루트 주소”를 반환하도록 만들고, 현재 노드가 그 주소를 left/right에 다시 저장합니다.

```
root->left = insert(root->left, key);
root->right = deleteNode(root->right, key);
```

9-5. 재귀 사고 훈련

- 현재 root가 NULL이면 무조건 끝나는가?
- 현재 root에서 무엇을 하고 난 다음 어느 자식으로 가는가?
- 재귀 호출이 돌아오면 반환값을 어디에 저장해야 하는가?
- 노드를 free했다면 그 자리를 대신할 포인터를 무엇으로 반환할 것인가?

10. 시간복잡도와 문제 읽는 법

자료구조 문제는 구현만 맞으면 끝이 아니라, 왜 어떤 구조를 쓰는지 이해해야 합니다. 가장 기본적인 척도가 Big-O입니다.

복잡도	직관	대표 예
$O(1)$	데이터 크기와 무관하게 거의 한 번에	Stack push/pop, List head 삽입
$O(\log n)$	문제를 절반씩 좁혀감	균형 잡힌 BST 탐색
$O(n)$	전체를 한 번 훑음	Linked List search, Tree traversal
$O(n^2)$	중첩 순회가 혼함	기본적인 이중 반복
$O(h)$	Tree에서 높이에 비례	BST search/insert/delete

10-1. BST는 항상 $O(\log n)$ 이 아니다

BST가 한쪽으로 치우치면 사실상 연결 리스트처럼 됩니다. 예를 들어 1,2,3,4,5를 순서대로 삽입하면 한쪽으로 길게 늘어진 트리가 되어 탐색이 $O(n)$ 에 가까워질 수 있습니다. 균형이 잘 잡힌 경우에는 $O(\log n)$ 에 가까운 탐색이 가능합니다.

10-2. 자료구조 선택 감각

요구	잘 맞는 구조
맨 앞에서 빠르게 넣고 빼기	Linked List / Stack
최근 데이터부터 꺼내기	Stack
먼저 온 순서대로 처리	Queue
계층 관계 표현	Tree
정렬된 조건으로 검색	BST
인덱스로 임의 접근	Array
양 끝 삽입/삭제	Deque 계열 고려

10-3. 과제 코드를 읽을 때 먼저 찾을 것

- 구조체 정의: Node의 필드가 무엇인지
- 초기값: head/top/front/rear/root가 무엇을 의미하는지
- 반환값: 함수가 int인지 Node *인지, 성공/실패를 어떻게 알리는지
- 경계조건: 빈 구조/한 개/마지막 원소/중복값
- 메모리 소유권: malloc한 것을 누가 free하는지

11. 자주 터지는 오류와 디버깅

자료구조 문제에서는 컴파일은 되지만 실행 중 잘못된 연결이 만들어지는 경우가 많습니다. 이 장은 그런 오류를 찾는 방법입니다.

11-1. 포인터 체크 5단계

- 이 포인터가 NULL일 수 있는가?
- 이 포인터가 현재 어떤 노드를 가리키는가?
- next/left/right를 바꾼 후 원래 연결을 잃지 않았는가?
- free한 포인터를 다시 사용하지 않는가?
- 함수에서 바뀐 head/root를 호출한 쪽이 반영받는가?

11-2. Linked List 디버깅 그림

변경 전
head->A->B->C->NULL

B 삭제 후 목표
head->A----->C->NULL

잘못된 예
A->B를 그대로 두고 B를 free
=>A->next가 해제된 메모리를 가리킴

11-3. BST 삭제 디버깅

삭제할 노드가 root인지, left subtree인지, right subtree인지 먼저 구분합니다. 그다음 자식 수를 확인합니다. 특히 자식 둘인 경우 “값만 복사했는지”, “실제 successor 노드도 삭제했는지” 둘 다 확인해야 합니다.

11-4. 무한 루프 체크

```
while (cur != NULL) {
    ...
    cur = cur->next; // 이동이 반드시 있어야 함
}
```

Tree의 재귀도 마찬가지입니다. base case가 없거나, 동일한 node를 계속 재호출하면 종료되지 않습니다.

11-5. 출력 디버깅

```
printf("cur=%p item=%d next=%p\n",
       (void*)cur, cur->item, (void*)cur->next);
```

주소 자체를 출력하면 “내가 생각한 노드와 실제 포인터가 맞는지” 빠르게 확인할 수 있습니다. 단, 역참조 전에 NULL인지 먼저 확인하세요.

11-6. 컴파일 경고를 적극적으로 보기

```
gcc -Wall -Wextra -std=c11 main.c -o main
```

경고는 “당장은 실행되지만 문제가 될 가능성이 있는 부분”을 알려주는 중요한 신호입니다. 자료구조 과제에서는 포인터 타입과 반환값 관련 경고를 특히 가볍게 넘기지 않는 것이 좋습니다.

12. 실전 구현 템플릿 모음

아래 템플릿은 “정답을 외우기 위한 코드”라기보다, 문제를 풀 때 구조를 잡는 뼈대입니다. 과제에서 제공된 구조체와 함수 이름이 있다면 그 정의에 맞게 바꿔 사용해야 합니다.

12-1. Linked List 순회 / 검색

```
Node *cur = head;
while (cur != NULL) {
    if (cur->item == target) {
        // 찾음
        break;
    }
    cur = cur->next;
}
```

12-2. Linked List prev/cur 패턴

```
Node *prev = NULL;
Node *cur = head;
while (cur != NULL && CONDITION) {
    prev = cur;
    cur = cur->next;
}
// cur: 현재 위치
// prev: cur의 바로 앞 노드
```

12-3. Stack 기본

push: top을 한 칸 올리고 저장
pop : top의 값을 사용하고 한 칸 내림

empty 조건을 먼저 처리한다.

12-4. Circular Queue 기본

```
nextIndex = (index + 1) % CAP;
```

```
empty: count == 0
full : count == CAP
```

```
enqueue: q[rear]=x; rear=nextIndex; count++
dequeue: x=q[front]; front=nextIndex; count--
```

12-5. Binary Tree 순회 3종

```
preorder(node) { root, left, right }
inorder(node) { left, root, right }
postorder(node) { left, right, root }
```

모든 경우 첫 줄에
if (node == NULL) return;
형태의 종료 조건을 둔다.

12-6. BST 검색/삽입 결정

```
if (key < root->key)
    left
else if (key > root->key)
    right
else
    found / duplicate / no-op
```

12-7. BST 삭제 의사코드

1. root가 NULL이면 종료
2. key가 작으면 left에서 삭제
3. key가 크면 right에서 삭제
4. 같으면
 - a) 왼쪽 없음 -> 오른쪽 반환
 - b) 오른쪽 없음 -> 왼쪽 반환

c) 둘 다 있음 :> 오른쪽 최소값을 복사하고 그 노드 삭제
5. 최종 root 반환

12-8. 함수가 head/root를 바꿔야 할 때

```
void func(Node **head) { ... }  
  
// 호출  
func(&head);  
  
// 또는  
Node *newRoot = insert(root, x);  
root = newRoot;
```

C언어 보충 통합본

13단원 빈칸 보충본 + 최종본 부족분 보충 완성본

두 부록을 한 권으로 합치고, 겹치는 항목은 한 곳에만 두었습니다.

최종본(80쪽)은 그대로 기본 교재입니다. 이 문서는 최종본에 없거나 코드가 약했던 부분만 모은 부록입니다.

부	내용
1부 13단원 빈칸	기본문법부터 파일입출력까지, 최종본에 없던 소항목
2부 서식·헤더·변환	%lld/%lf, 헤더 지도, strtol, assert
3부 구현 뼈대	리스트/환형/스택/큐/BST 완성 코드, Makefile
4부 연습	문항+정답, 시험 전 체크리스트

- 1부 01 기본문법 (이름, 주석, 이스케이프 나머지)
- 1부 02~13 입출력 / 변수 / 연산 / 조건 / 반복 / 함수 / 배열 / 포인터 3장 / 사용자형 / 파일
- 2부 서식 표, 헤더, 문자열 +1, strtol, assert
- 3부 리스트 삽입·삭제, 환형 순회, 스택, 원형 큐, BST 삭제, Makefile
- 4부 연습 10문항, 체크리스트

1부. 13단원 빈칸

01. 기본문법

규칙	예
영문, 숫자, 밑줄만. 숫자로 시작 불가	sum1 가능 / 1sum 불가
대소문자 구분	Count 와 count 는 다른 이름
키워드는 이름 불가	int, return, if

```
/* 여러 줄 주석 */
int n = 10; // 한 줄 주석
n + 1;      /* 식. 결과가 저장되지 않음 */
n = n + 1;  /* 문장. 값이 변수에 저장됨 */
```

이스케이프 — 최종본 표에 없는 것만

최종본 6쪽에 이미 있는 것: \n \t \r \b \\ \' \" \0. 아래는 그 표에 없던 것입니다.

표기	뜻
\a	경고음(벨)
\v	세로 탭
\f	폼 피드(페이지 넘김)
\ooo	8진수 문자. \101 은 A
\xhh	16진수 문자. \x41 은 A

```
printf("A\a\n"); /* 경고음이 날 수 있음 */
printf("\x41\n"); /* A */
printf("\101\n"); /* A, 8진수 101 */
/* \n 은 화면 줄바꿈, \0 은 문자열 끝. 둘은 다름 */
```

02. 표준 입출력

함수	하는 일	주의
putchar(c)	문자 하나 출력	
getchar()	문자 하나 입력	끝이면 EOF
puts(s)	문자열 + 줄바꿈	개행을 자동으로 붙임
fgets(buf, n, stdin)	한 줄 입력	크기를 같이 넘김
gets	쓰지 않음	크기를 모름

```
int n;
char line[32];
scanf("%d", &n);    /* 숫자만 읽고 엔터는 남음 */
getchar();         /* 남은 엔터 제거 */
fgets(line, 32, stdin);

printf("%5d\n", 7); /* 오른쪽 정렬 5칸 */
printf("%-5d\n", 7); /* 왼쪽 정렬 */
printf("%.2f\n", 3.1416);
```

03. 변수

형	흔한 크기(64비트)	메모
char / short / int	1 / 2 / 4	int 크기는 환경마다 다를 수 있음
long / long long	4또는8 / 8	%ld / %lld
float / double	4 / 8	실수는 보통 double

```
int a;          /* 지역 변수. 값이 정해져 있지 않음 */
printf("%d", a); /* 위험 */
const int MAX = 100;
/* MAX = 200; 은 안 됨 */
```

부호가 다른 비교

int 와 unsigned 를 같이 비교하면 int 쪽이 unsigned 로 바뀌는 경우가 많습니다. 형을 맞추고 비교하세요.

04. 연산자

```
int *p = NULL;
if (p != NULL && *p == 10) {} /* 단락 평가. NULL 이면 *p 안 봄 */

int max = (a > b) ? a : b;    /* 삼항 */
n += 2;                      /* n = n + 2 */
printf("%zu\n", sizeof n);   /* sizeof 는 함수가 아님 */
printf("%zu\n", sizeof(int));
```

05. 조건문

```
switch (cmd) {
    case 1:
        printf("one\n");
        /* break 없으면 case 2 도 실행됨 */
    case 2:
        printf("two\n");
        break;
    default:
        printf("other\n");
}
/* 실수는 == 로 직접 비교하지 말 것. 0.1+0.2 가 0.3 이 아닐 수 있음 */
```

06. 반복문

```
for (;;) { if (done) break; } /* 무한 루프 */

for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++)
        printf("%d ", i * 3 + j);
    printf("\n");
}
/* 0부터 n-1: i < n    1부터 n: i <= n */
```

07. 함수

```
int add(int a, int b); /* 원형 */

int main(void) {
    printf("%d\n", add(2, 3));
    return 0;
}

int add(int a, int b) { return a + b; }

void hello(void) { printf("hi\n"); }

int abs_int(int n) {
    if (n < 0) return -n;
    return n;
}
```

08. 배열 · 문자열 칸

```
int a[5] = {1, 2}; /* 나머지 0 */
char words[3][8] = {"cat", "dog", "bird"};
char *names[3] = {"Kim", "Lee", "Park"};

char ok[4] = "abc"; /* a b c 와 끝문자 */
char *src = "pintos";
char *bad = malloc(strlen(src)); /* 위험: 끝문자 자리 없음 */
char *safe = malloc(strlen(src) + 1); /* 안전 */
```

09~11. 포인터 / 배열 / 함수

```

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
int x = 3, y = 7;
swap(&x, &y);

void *vp = &x;
int *pi = (int *)vp;

void fill(int a[], int n) { /* int *a 와 같음 */
    int i;
    for (i = 0; i < n; i++) a[i] = i;
}

int *make(int n) {
    int *p = malloc(sizeof(int));
    if (p == NULL) return NULL;
    *p = n;
    return p;          /* heap. 호출한 쪽이 free */
}

void print2(int m[][3], int rows) { /* 열 크기는 적어야 함 */
    int i, j;
    for (i = 0; i < rows; i++)
        for (j = 0; j < 3; j++)
            printf("%d ", m[i][j]);
}

```

두 줄의 차이

char *ps[3] 은 주소 세 개. char buf[3][8] 은 문자 칸 24개. 지역 변수 주소 return &local; 은 함수가 끝나면 사라집니다.

12. 사용자 정의 자료형

```

struct Point { int x; int y; };
struct Rect { struct Point a; struct Point b; };
struct Rect r;
r.a.x = 0;

void move(struct Point p) { p.x += 1; } /* 복사본 */
void move_p(struct Point *p) { p->x += 1; } /* 원본 */

struct A { char c; int n; };
/* sizeof(struct A) 는 5가 아닐 수 있음. 패딩 */

enum Color { RED = 1, GREEN = 2, BLUE = 4 };

```

13. 파일입출력

```
FILE *fp = fopen("out.txt", "w");
if (fp == NULL) { perror("fopen"); return 1; }
fprintf(fp, "%d %s\n", 10, "cat");
fclose(fp);

fp = fopen("out.txt", "r");
if (fp == NULL) return 1;
int n; char s[20];
fscanf(fp, "%d %19s", &n, s);
fclose(fp);

int ch;
while ((ch = fgetc(in)) != EOF)
    fputc(ch, stdout);

fseek(f, 0, SEEK_END);
long size = ftell(f);
rewind(f);
```

모드	덧붙일 점
w	있으면 내용을 지우고 다시 씀
a	끝부터 이어 씀
r+	읽기+쓰기. 파일이 이미 있어야 함
w+	읽기+쓰기. 기존 내용을 지움

2부. 서식 · 헤더 · 변환

최종본 표의 %d %u %ld %f %c %s %p %x 와 보충 %o %zu 위에, 과제에서 자주 나오는 나머집니다.

서식	대상	메모
%lld / %llu	long long	64비트 정수
%f	printf 의 double	float 도 printf 에선 double 로 승격
%lf	scanf 의 double	출력의 %f 와 짝이 다름
%e / %g	double	지수 / 자동 선택
%zu	size_t	sizeof 결과
%p	void*	(void*)로 캐스팅

```
long long big = 1234567890123LL;
double x = 3.14159;
printf("%lld\n", big);
printf("%.2f %e %g\n", x, x, x);
scanf("%lf", &x); /* double 입력 */
```

자주 틀리는 짝

printf 에서 double 은 %f, scanf 에서 double 은 %lf. scanf("%f", &d) 로 double 을 받으면 정의되지 않은 동작입니다.

표준 헤더

헤더	대표
stdio.h	printf, scanf, fopen, fgetc, perror
stdlib.h	malloc, free, strtol, qsort, exit
string.h	strlen, strcpy, strcmp, memcpy
stddef.h / stdint.h / stdbool.h	size_t, NULL / int32_t / bool
assert.h / ctype.h / math.h / errno.h	assert / isdigit / sqrt(-lm) / errno

strtol / assert

```
char *end;
errno = 0;
long value = strtol(argv[1], &end, 10);
if (end == argv[1] || *end != '\0') {
    fprintf(stderr, "not a number\n");
    return 1;
}
if (errno != 0) { perror("strtol"); return 1; }

#include <assert.h>
assert(ptr != NULL); /* 내부 약속. 사용자 입력 검사용 아님 */
```

3부. 구현 뼈대

최종본은 패턴·의사코드 중심입니다. 과제 starter 이름만 바꿔 넣으면 되는 최소 코드입니다.

연결 리스트

```

typedef struct Node {
    int item;
    struct Node *next;
} Node;

Node *node_new(int item) {
    Node *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->item = item;
    n->next = NULL;
    return n;
}

void push_front(Node **head, int item) {
    Node *n = node_new(item);
    if (n == NULL) return;
    n->next = *head;
    *head = n;
}

void delete_value(Node **head, int item) {
    Node *prev = NULL, *cur = *head;
    while (cur != NULL && cur->item != item) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) return;
    if (prev == NULL) *head = cur->next;
    else prev->next = cur->next;
    free(cur);
}

void list_free(Node **head) {
    Node *cur = *head;
    while (cur != NULL) {
        Node *next = cur->next; /* free 전에 다음을 저장 */
        free(cur);
        cur = next;
    }
    *head = NULL;
}

void print_circular(Node *head) {
    Node *cur = head;
    if (cur == NULL) return;
    do {
        printf("%d ", cur->item);
        cur = cur->next;
    } while (cur != head);
}

```

위험한 한 줄

`free(cur); cur = cur->next;` 는 use-after-free. 반드시 `Node *next = cur->next;` 를 먼저 적습니다.

배열 스택 / count 원형 큐

```
#define MAX 100
int stack[MAX], top = -1;
int push(int x) {
    if (top == MAX - 1) return 0;
    stack[++top] = x;
    return 1;
}
int pop(int *out) {
    if (top == -1) return 0;
    *out = stack[top--];
    return 1;
}

#define CAP 8
int q[CAP], front = 0, rear = 0, count = 0;
int enqueue(int x) {
    if (count == CAP) return 0;
    q[rear] = x;
    rear = (rear + 1) % CAP;
    count++;
    return 1;
}
int dequeue(int *out) {
    if (count == 0) return 0;
    *out = q[front];
    front = (front + 1) % CAP;
    count--;
    return 1;
}
```

원형 큐 정의

count 없이 $(rear+1)\%CAP == front$ 를 full 로 보는 구현도 있습니다. 과제 starter 와 정의를 섞지 마세요.

BST 삭제

3부 이어서 · BST 삭제 · Makefile

원본 84쪽에서 연습·체크리스트만 빼고, 구현 코드를 그대로 남긴 페이지입니다.

BST 삭제

```
typedef struct TreeNode {
    int key;
    struct TreeNode *left, *right;
} TreeNode;

TreeNode *tree_min(TreeNode *root) {
    while (root && root->left) root = root->left;
    return root;
}

TreeNode *bst_delete(TreeNode *root, int key) {
    if (root == NULL) return NULL;
    if (key < root->key)
        root->left = bst_delete(root->left, key);
    else if (key > root->key)
        root->right = bst_delete(root->right, key);
    else {
        if (root->left == NULL) {
            TreeNode *r = root->right; free(root); return r;
        }
        if (root->right == NULL) {
            TreeNode *l = root->left; free(root); return l;
        }
        TreeNode *succ = tree_min(root->right);
        root->key = succ->key;
        root->right = bst_delete(root->right, succ->key);
    }
    return root;
}
```

자식이 0개면 free 후 NULL. 1개면 남은 자식만 연결. 2개면 오른쪽 최솟값을 복사한 뒤 그 노드를 삭제합니다.

Makefile

```
CC = gcc
CFLAGS = -std=c17 -Wall -Wextra -g

app: main.o list.o
$(CC) $(CFLAGS) main.o list.o -o app
main.o: main.c list.h
$(CC) $(CFLAGS) -c main.c
list.o: list.c list.h
$(CC) $(CFLAGS) -c list.c
clean:
rm -f *.o app
```

환형 단일 연결 리스트

마지막 노드의 next가 NULL이 아니라 다시 head를 가리킵니다. 끝이 없는 원입니다.

상태	모습	검사
빈 리스트	head == NULL	head가 NULL

상태	모습	검사
노드 1개	그 노드의 next가 자기 자신	head->next == head
노드 여러 개	마지막 next == head	한 바퀴 돌면 다시 head

노드와 한 칸 삽입(앞에 넣기)

```
typedef struct Node {
    int item;
    struct Node *next;
} Node;

/* 빈 리스트에 첫 노드를 넣을 때: next를 자기 자신으로 */
Node *circular_insert_front(Node *head, int item) {
    Node *n = malloc(sizeof *n);
    if (n == NULL) return head;
    n->item = item;
    if (head == NULL) {
        n->next = n;
        return n;
    }
    /* 마지막 노드를 찾아 n을 head 앞에 끼운다 */
    Node *tail = head;
    while (tail->next != head)
        tail = tail->next;
    n->next = head;
    tail->next = n;
    return n; /* 새 head */
}
```

순회 — while (cur != NULL) 를 쓰면 끝나지 않습니다

```
void print_circular(Node *head) {
    if (head == NULL) return;
    Node *cur = head;
    do {
        printf("%d ", cur->item);
        cur = cur->next;
    } while (cur != head);
}
```

선형 리스트는 NULL이 끝입니다. 환형은 끝이 없으므로 한 번 실행한 뒤 cur != head 가 될 때까지 돕니다. do-while이 기본 패턴입니다.

자주 하는 실수

실수	결과	고치는 방법
while (cur != NULL)	무한 루프	do { ... } while (cur != head)
마지막 next를 NULL로 둠	더 이상 환형이 아님	tail->next = head 유지
노드 1개 삭제 시 head만 NULL	자기 자신 next를 못 끊음	혼자면 free 후 head = NULL

환형 이중 연결 리스트

각 노드가 prev와 next를 가지고, 양쪽이 모두 원입니다. head의 prev는 마지막 노드, 마지막 노드의 next는 head입니다.

```
typedef struct DNode {
    int item;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

왜 쓰나

비교	환형 단일	환형 이중
뒤로 가기	한 바퀴를 더 돌아야 함	prev로 바로 이동
중간 삭제	앞 노드를 따로 기억	cur->prev, cur->next만으로 가능
메모리	포인터 1개	포인터 2개

삽입 · 삭제 핵심

```
/* 노드 1개짜리 원 만들기 */
DNode *d_one(int item) {
    DNode *n = malloc(sizeof *n);
    n->item = item;
    n->prev = n->next = n;
    return n;
}

/* cur 뒤에 n 끼우기 */
void d_insert_after(DNode *cur, DNode *n) {
    n->next = cur->next;
    n->prev = cur;
    cur->next->prev = n;
    cur->next = n;
}

/* cur 빼기. 혼자면 NULL 반환 */
DNode *d_erase(DNode *head, DNode *cur) {
    if (cur->next == cur) { /* 혼자 */
        free(cur);
        return NULL;
    }
    cur->prev->next = cur->next;
    cur->next->prev = cur->prev;
    if (head == cur) head = cur->next;
    free(cur);
    return head;
}
```

순서는 항상 이웃의 포인터를 먼저 고친 다음 free 합니다. 양방향이므로 앞 노드를 찾기 위해 리스트를 다시 돌 필요가 없습니다.

양방향 순회

```
/* 시계 방향 */
DNode *cur = head;
do { /* cur 사용 */ cur = cur->next; } while (cur != head);

/* 반시계 방향 */
cur = head;
do { /* cur 사용 */ cur = cur->prev; } while (cur != head);
```

환형 큐 (원형 큐)

리스트의 '환형'은 노드가 원을 이루는 것이고, 큐의 '환형'은 배열 인덱스를 모듈로로 다시 처음으로 보내는 것입니다. 둘은 다른 자료구조입니다.

이름	무엇이 원인가	대표 용도
환형 단일 리스트	노드의 next	원형 버퍼, 라운드로빈
환형 이중 리스트	next와 prev	앞뒤로 이동하는 작업 큐
환형 큐	배열 인덱스 % CAP	FIFO, 고정 크기 버퍼

count로 빈/가득 구분하기

```
#define CAP 8
int q[CAP], front = 0, rear = 0, count = 0;

int enqueue(int x) {
    if (count == CAP) return 0; /* full */
    q[rear] = x;
    rear = (rear + 1) % CAP;
    count++;
    return 1;
}

int dequeue(int *out) {
    if (count == 0) return 0; /* empty */
    *out = q[front];
    front = (front + 1) % CAP;
    count--;
    return 1;
}
```

한 칸을 비워 두고 구분하는 방법

count를 안 쓰면, 빈 큐와 가득 찬 큐에서 `front == rear` 가 같아질 수 있습니다. 그래서 칸을 하나 비워 두고 `(rear + 1) % CAP == front` 를 가득 참으로 보는 구현이 있습니다. 실제 저장량은 `CAP - 1` 입니다.

```
/* count 없이: 한 칸을 희생 */
int is_empty(void) { return front == rear; }
int is_full(void) { return (rear + 1) % CAP == front; }
```

과제 starter 코드가 count를 쓰는지, 한 칸을 비우는지 먼저 확인하세요. 두 정의를 섞으면 full/empty 판정이 깨집니다.

연결 리스트로 만든 큐

head를 front, tail을 rear로 두면 enqueue-dequeue가 거의 $O(1)$ 입니다. 배열 환형 큐는 크기가 고정이고, 리스트 큐는 늘어날 수 있는 대신 malloc이 필요합니다.

C에서 NaN과 INF (파이썬과 비교)

파이썬의 `float('nan')`, `float('inf')` 와 같은 값이 C에도 있습니다. C99부터 `<math.h>` 의 `NAN`, `INFINITY` 를 씁니다. 컴파일할 때 보통 `-lm` 이 필요합니다.

파이썬	C (math.h)	뜻
<code>float('nan') / math.nan</code>	<code>NAN</code>	숫자가 아님 (Not a Number)
<code>float('inf') / math.inf</code>	<code>INFINITY</code>	양의 무한대
<code>-float('inf')</code>	<code>-INFINITY</code>	음의 무한대

파이썬	C (math.h)	뜻
math.isnan(x)	isnan(x)	NaN인지 검사
math.isinf(x)	isinf(x)	무한대인지 검사
math.isfinite(x)	isfinite(x)	유한한 수인지 검사

만드는 방법과 출력

```
#include <stdio.h>
#include <math.h>

int main(void) {
    double nan_v = NAN; /* 또는 nan("") */
    double inf_v = INFINITY; /* 또는 1.0 / 0.0 */
    double ninf = -INFINITY;

    printf("%f %f %f\n", nan_v, inf_v, ninf);
    /* 보통: nan inf -inf */

    printf("isnan %d\n", isnan(nan_v)); /* 1 */
    printf("isinf %d\n", isinf(inf_v)); /* 1 */
    return 0;
}
/* gcc main.c -o main -lm */
```

가장 중요한 규칙: NaN은 자기 자신과도 같지 않다

```
double x = NAN;
if (x == NAN) {} /* 항상 거짓. 이렇게 비교하지 말 것 */
if (isnan(x)) {} /* 올바른 검사 */

/* 파이썬도 같다: float('nan') == float('nan') 은 False */
```

0.0 / 0.0 은 보통 NaN, 1.0 / 0.0 은 보통 +INF 입니다. 정수 0 / 0 은 부동소수점이 아니라서 프로그램이 죽을 수 있습니다. 실수 나눗셈과 정수 나눗셈을 섞지 마세요.

어디서 나오나

연산	결과 예
0.0 / 0.0, sqrt(-1.0), log(-1.0)	NaN
1.0 / 0.0, exp(큰 수)	+INF
-1.0 / 0.0	-INF
INF - INF, INF / INF	NaN
아무 수 + NaN	NaN (전염)

입력값을 검사할 때는 isnan, isinf, isfinite 를 먼저 쓰는 편이 안전합니다. 비교 연산 == 으로는 NaN을 잡을 수 없습니다.